Package pyliferisk

November 11, 2026

Type:	Python Package
Version:	1.13
Author:	Francisco Gárate Santiago
Contact:	fgarate@ucm.es
License:	GPLv3
Repository:	`https://github.com/franciscogarate/pyliferisk`

Abstract: A python library for life actuarial calculations. Simple, powerful and easy-to-use.

This document aims to be the only necessary and authoritative source of information about pyliferisk, usable as a comprehensive reference, an user guide, and tutorial all-in-one. It also includes a sample of illustrative and common examples of actuarial calculations.

Pyliferisk requires Python 3 (version 3.8 or later) and has no dependencies other than the Python Standard Library, making it amazingly fast.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. The author does not take any legal responsibility for the accuracy, completeness, or usefulness of the information herein.

Contents

1 Introduction 3

2 Pyliferisk: List of formulas 6
- 2.1 Biometric functions: Class MortalityTable . . . 6
  - 2.1.1 method `.qx[x]` . . . 7
  - 2.1.2 method `.lx[x]` . . . 7
  - 2.1.3 method `.w` . . . 7
  - 2.1.4 method `mt.dx[x]` . . . 7
  - 2.1.5 method `.ex[x]` . . . 8
- 2.2 Actuarial Present Value: Class Actuarial . . . 9
  - 2.2.1 formula `Ax()` . . . 9
  - 2.2.2 formula `Axn()` . . . 9
  - 2.2.3 formula `qAx()` . . . 10
  - 2.2.4 formula `qAxn()` . . . 10
  - 2.2.5 formula `AExn()` . . . 10
- 2.3 Annuity formula . . . 13
  - 2.3.1 function `annuity()` . . . 13
- 2.4 Pure endowment: Deferred capital . . . 18
  - 2.4.1 formula `nEx()` . . . 18
- 2.5 Commutation factors . . . 19
- 2.6 Other functions . . . 19
- 2.7 Help (Documentation strings) . . . 21

3 Mortality tables 22
- 3.1 Adding tables . . . 22
  - 3.1.1 Probability of dying tables (qx) . . . 22
  - 3.1.2 Number of surviving tables (lx) . . . 22
- 3.2 Class .view() . . . 24

4 Examples 25
- 4.1 Example 1. Cashflow calculation: Annuity-immediate Geometrically increasing . 25
- 4.2 Example 2. Drawing graph with matplotlib . . . 26
- 4.3 Example 3. Obtain data from plain text file . . . 27
- 4.4 Example 4. Obtain the Risk-free rate from MS Excel . . . 28
- 4.5 Example 5. Cashflow Calculation Reserving . . . 29
- 4.6 Example 6. Import a mortality table (lx) using pandas . . . 31
- 4.7 Example 7. Cashflows in pandas . . . 33

5 Changes in version 1.13 35

6 Books 35

7 Acknowledgements 35

1 Introduction

Pyliferisk is an open library written in Python for life and actuarial calculation contracts, based on commonly used methodologies among actuaries (International Actuarial Notation).

This library is able to cover all life contingencies risks (since the actuarial formulas follow the International Actuarial Notation), as well as to support the main insurance products: Term Life, Whole Life, Annuities and Universal Life. Additionally, the library can be easily tailored to any particular or local specifications, since Python is a very intuitive language.

It is ideal not only for academic purposes but also for professional use by actuaries (implementation of premiums and reserves modules) or by auditors (validation of reserves or capital risk models such as parallel runs).

This library is distributed as a single file module and has no dependencies other than the Python Standard Library, making it amazingly fast. Additionally, the package includes several life mortality tables (`pyliferisk.mortalitytables`), mainly extracted from academic textbooks. Nevertheless, additional libraries as Numpy or Pandas may be required for increasing functionality, such as cash flow operations, random number generation, interpolation, etc.

You can find also examples for different life contracts in the examples section.

Why Python?

Because computing plays an important role in the actuarial profession, but actuaries are not programmers. Python is friendly and easy to learn.

Nowadays, programming is becoming an indispensable skill for actuaries. Python is a clear, readable syntax, powerful and fast language. Easy to learn, especially when you are not used to coding. This language lets you write quickly the code you need, without cumbersome rules or variable predefined tasks. It is clear, forget ending with commas and using curly brackets in functions.

For European actuaries, Solvency II opens a big opportunity. The new requirements transform into agility, transversality, and auditability. The internal model is not only software, but it should also be an internal process used extensively where all parts must walk hand in hand.

Python version

Pyliferisk requires Python 3 (version 3.8 or later). Python 2.7 is no longer supported.

Disclaimer: qx is expressed per unit since version 1.13

Breaking change. Up to version 1.12, `mt.qx[x]`, `qx()` and `px()` were expressed per mille (the true probability was qx/1000). Since version 1.13 they are expressed per unit (between 0 and 1), so `mt.qx[x]` is the true probability.

This library is mainly used for academic purposes, and the aim of this change is to align it with the reference bibliography, where q_x and p_x are probabilities between 0 and 1.

Code written for previous versions must be reviewed: any division by 1000 (for instance `mt.qx[x] / 1000`) must be removed, otherwise the results will be wrong without any error message. The mortality tables included in `mortalitytables.py` are still stored per mille and they are converted automatically when the table is loaded. See section 5 for the complete list of changes.

Potential uses

This library may be used in tariff processes, in the design phase of new products such as profit testing or estimation of future benefits. Other uses include:

- Auditing purposes tool
- Assumption calibrations, back-testing, etc..
- Replicate the main calculations of the internal model for implementation in pricing, product approval, reserving, etc..
- Perform small reports (output format may be xml, xls, etc...)

If you find something that Python cannot do, or if you need the performance advantage of low-level code, you can write or reuse extension modules in C or C++.

Installation

Once Python is running, just install this library with `pip install`

```
> pip install pyliferisk
```

Then, to import this library in projects is automatic as usually:

Option 1:

```
from pyliferisk import *
from pyliferisk.mortalitytables import *

tariff = MortalityTable(nt=GKM95)
```

Option 2:

```
import pyliferisk as life
import pyliferisk.mortalitytables as mort

tariff = life.MortalityTable(nt=mort.GKM95)
```

Option 3, if only like to use specific functions:

```
from pyliferisk import MortalityTable
from pyliferisk.mortalitytables import GKM95

tariff = MortalityTable(nt=GKM95)
```

Update library

Run this command from terminal:

```
> pip install pyliferisk --upgrade
```

License

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or any later version.

2 Pyliferisk: List of formulas

The names of the formulas follow the International Actuarial Notation and are easily guessable (qx, lx...) with a few exceptions regarding special characters.

2.1 Biometric functions: Class MortalityTable

The Instance Variables for the `MortalityTable()` class are:

- `nt` = The actuarial table used to perform life contingencies calculations. Syntax: nt = GKM95 (Note: GKM95 must be created previously or be included in mortalitytables.py)

  Example:

```
tariff = MortalityTable(nt=GKM95)
```

- `perc` = Optional variable to indicate the percentage of mortality to be applied. Syntax: perc=85 . Variable `perc` can be omitted, in this case it will be 100 by default.

  Example:

```
experience = MortalityTable(nt=GKM95, perc=85)
```

- `lx` = Optional list with the number of survivors at each age, as an alternative to `nt` (see section Adding tables).
- `qx` = Optional list with the probabilities of dying at each age, expressed per unit (between 0 and 1), as an alternative to `nt`. A `ValueError` is raised if any value is outside that range (for instance, a list expressed per mille).

Conventions:

- Probabilities are per unit. Internally the library works with probabilities per unit (between 0 and 1): `mt.qx[x]` is the true probability. The source tables included in `mortalitytables.py` are expressed per mille and they are converted when the table is loaded. The per mille figures are only offered as presentation of the results, with `mt.view(var='qx')` (or simply multiplying by 1000).
- `MortalityTable` is a purely biometric class and it does not discount: the interest rate `i` belongs to the class `Actuarial`. If `i` is passed to `MortalityTable`, it is ignored and a `DeprecationWarning` is issued.

Once define the variables for your mortality table, all available biometric functions are the following:

2.1.1 method `.qx[x]`

Description	Returns the probability that a life aged x dies before 1 year, expressed per unit (between 0 and 1)
Actuarial notation	q_x
Usage	`mt.qx[x]`
Args	x: the age as integer number.
Example	`tariff = MortalityTable(nt=PASEM2020_M_NoRel_1orden)` `tariff.qx[50]`

2.1.2 method `.lx[x]`

Description	Returns the number of survivors at beginning of age x
Actuarial notation	l_x
Usage	`mt.lx[x]`
Args	x: the age as integer number.
Example	`tariff.lx[50]`

2.1.3 method `.w`

Description	ultimate age (lw = 0)
Actuarial notation	w
Usage	`mt.w`
Example	`tariff.w`

2.1.4 method `mt.dx[x]`

Description	Returns the number of dying at beginning of age x
Actuarial notation	d_x
Usage	`mt.dx[x]`
Args	x: the age as integer number.
Example	`tariff.dx[50]`

2.1.5 method `.ex[x]`

Description	Returns the curtate expectation of life (complete years only): $e_x = \sum_{k \geq 1} l_{x+k} \,/\, l_x$ The half year of the complete expectation is not added ($e_x^{\circ} \approx e_x + 0.5$)
Actuarial notation	e_x
Usage	`mt.ex[x]`
Args	self: the mortality table x: the age as integer number.
Example	`tariff.ex[50]`

Example:

Print the omega (limiting age) of the both mortality tables and the qx at 50 years-old:

```
from pyliferisk import MortalityTable
from pyliferisk.mortalitytables import PASEM2020_M_NoRel_1orden,
    GKM95

tariff = MortalityTable(nt=PASEM2020_M_NoRel_1orden)
experience = MortalityTable(nt=GKM95, perc=85)

# Print the omega (limiting age) of the both tables:
print(tariff.w)
print(experience.w)

# Print the qx at 50 years old:
print(tariff.qx[50])
print(experience.qx[50])
```

Return the following results:

```
108
121
0.002310560461
0.003662395
```

2.2 Actuarial Present Value: Class Actuarial

The Present Value of the benefit payment is a function of time of death given a survival model and an interest rate.

The class `Actuarial` inherits from `MortalityTable`: it builds the same biometric variables (`qx, lx, dx, ex, w`), it accepts the same arguments (`nt, lx, qx, perc`) and it shares the method `.view()`, adding only the commutation vectors (`Dx, Nx, Cx, Mx`) calculated with the interest rate `i`. Therefore, every `Actuarial` instance can be used wherever a `MortalityTable` is expected.

The Instance Variables for the `Actuarial()` class are:

- `nt` = The actuarial table used to perform life contingencies calculations.
  Syntax: nt=GKM95 (Note: GKM95 must be included in mortalitytables.py)
- `i` = interest rate. The effective rate of interest, namely, the total interest earned in a year. It is mandatory: a `ValueError` is raised if it is omitted.
  Syntax: i=0.02
- `perc` = Optional variable to indicate the percentage of mortality to be applied.
  Syntax: perc=85 . Variable `perc` can be omitted, in this case, it will be 100 by default.

```
tariff = Actuarial(nt=GKM95, i=0.05)
```

2.2.1 formula `Ax()`

Description	Returns the Expected Present Value (EPV) of a whole life insurance (i.e. net single premium). It is also commonly referred to as the Actuarial Value or Actuarial Present Value.
Actuarial notation	A_x
Usage	`Ax(mt, x)`
Args	mt: Mortality table. x: the age as integer number.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `Ax(mt, 50)`

2.2.2 formula `Axn()`

Description	Returns the EPV (net single premium) of a term insurance.
Actuarial notation	$A^1_{x:\overline{n}\rceil}$
Usage	`Axn(mt, x, n)`
Args	mt: Mortality table. x: the age as integer number. n: period in years.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `Axn(mt, 50, 10)`

2.2.3 formula `qAx()`

Description	This function evaluates the APV of a geometrically increasing annual annuity-due.
Actuarial notation	${}_qA_x$
Usage	`Axn(mt, x, q)`
Args	mt: Mortality table. x: the age as integer number. q: increase rate.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `Axn(mt, 50, 0.03)`

2.2.4 formula `qAxn()`

Description	This function evaluates the APV of a geometrically increasing Term insurance.
Actuarial notation	${}_qA_{x:\overline{n}\rceil}$
Usage	`qAxn(nt, x, n, q)`
Args	mt: Mortality table. x: the age as integer number. n: period in years. q: increase rate.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `Axn(mt, 50, 10, 0.03)`

2.2.5 formula `AExn()`

Description	Returns the EPV of an endowment insurance. An endowment insurance provides a combination of a term insurance and a pure endowment
Actuarial notation	$A_{x:\overline{n}\rceil}$
Usage	`AExn(mt, x, n)`
Args	mt: Mortality table. x: the age as integer number. n: period in years.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `AExn(mt, 50, 10)`

Syntax:

Notation	Description	Syntax
A_x	whole-life death insurance	`Ax(nt, x)`
$A^1_{x:\overline{n}\rceil}$	Term insurance	`Axn(nt, x, n)`
$A_{x:\overline{n}\rceil}$	Endowment insurance	`AExn(nt, x, n)`
${}_qA_x$	Increasing whole-life	`qAx(nt, x, n)`
${}_qA_{x:\overline{n}\rceil}$	Increasing Term insurance	`qAxn(nt, x, n, q)`

Examples:

1) A whole-life single premium:

```
from pyliferisk import Actuarial, Ax
from pyliferisk.mortalitytables import GKM95

mt = Actuarial(nt=GKM95, i=0.02)
x = 50          #age
C = 1000        #capital

print(Ax(mt, x) * C)
```

```
583.2765526571226
```

2) A term insurance single premium:

```
from pyliferisk import Actuarial, Axn
from pyliferisk.mortalitytables import GKM95

mt = Actuarial(nt=GKM95, i=0.03)
x = 40          #age
n = 20          #horizon
C = 10000       #capital

print(Axn(mt, x, n) * C)
```

```
636.6691728668218
```

3) Example 2 with cashflow approach:

```
from pyliferisk import MortalityTable
from pyliferisk.mortalitytables import GKM95
import numpy as np

mt = MortalityTable(nt=GKM95)
x = 40          #age
n = 20          #horizon
C = 10000       #capital
i = 0.03        #interest rate

payments = []
for t in range(0, n):
   payments.append((mt.lx[x+t] - mt.lx[x+t+1]) / mt.lx[x] * C)

discount_factor = []
for y in range(0, n):
   discount_factor.append(1 / (1 + i) ** (y + 0.5))

print(np.dot(discount_factor, payments).round(2))
```

```
646.15
```

To print the results year by year:

```
print('{0:5} {1:10} {2:10}'.format(' t','factor','payment'))

for t in range(0,n):
        print('{0:2} {1:10} {2:10}'.format(t, np.around(
           discount_factor[t],5), np.around(payments[t],4)))
```

```
 t    factor     payment
 0    0.98533     18.694
 1    0.95663    19.9456
 2    0.92877    21.3621
 3    0.90172    22.9574
 4    0.87545    24.7629
 5    0.84995     26.815
 6     0.8252    29.1475
 7    0.80116    31.7959
 8    0.77783    34.7884
 9    0.75517    38.1576
10    0.73318    41.9304
11    0.71182    46.1285
12    0.69109    50.7779
13    0.67096    55.8959
14    0.65142    61.4878
15    0.63245    67.5536
16    0.61402    74.0921
17    0.59614     81.095
18    0.57878    88.5512
19    0.56192    96.4435
```

2.3 Annuity formula

A life annuity refers to a series of payments to an individual as long as the individual is alive on the payment date. It may be temporary or payable for whole-life. The payment intervals may commence immediately or deferred. The payment may be due at the beginnings of the intervals (annuity due) or at the end (annuity immediate).

2.3.1 function `annuity()`

Description	Returns the actuarial present value of annuity payments	
Usage	`annuity(mt, x, n, p, m=1, deferred=0, increase=None)`	
Args		
1.	mt	the mortality table (an `Actuarial` instance)
2.	x	The age of the insured
3.	n	The horizon (term of insurance) in years or payment duration or
	or `'w'`	w as whole-life. Also 99 is defined as whole-life.
4.	p = 0	annuity-due (payments at the beginning of each period)
	or p = 1	annuity-immediate (payments at the end of each period)
optional args:		
–	m	Number of fractional payments per period. If missing, m is set as 1
keyword args:		
–	deferred	`deferred=d`: the deferring period in years (0 by default)
–	increase	`increase=['a', q]`: arithmetically increasing (unitary)
		`increase=['g', q]`: geometrically increasing at rate q
		If missing, it is `None` (level annuity)

The deferring period and the increase are passed as named parameters:

```
mt = Actuarial(nt=GKM95, i=0.02)
annuity(mt, 30, 10, 0, deferred=5)
annuity(mt, 60, 'w', 0, deferred=5, increase=['g', 0.02])
```

```
8.18406119600546
15.135655868886642
```

Notes:

- A `ValueError` is raised if `p` is neither 0 nor 1, or if `increase` is not `['a', q]` or `['g', q]`. The same error is raised if `deferred` is not a non-negative integer, or if the deferring period or the increase are given twice (positionally and by name). No result is returned with invalid arguments.
- The former positional syntax `annuity(mt, x, n, p, m, ['a'/'g', q], -d)`, where the deferring period was passed as a negative number, is deprecated. It is still accepted, but it issues a `DeprecationWarning`.
- A `NotImplementedError` is raised for deferred temporary arithmetically increasing annuities (not implemented yet).
- Geometrically increasing annuities and insurances (`increase=['g', q]` and the functions `qax, qaax, qaxn, qaaxn, qtax, qtaax, qtaxn, qtaaxn, qAx, qAxn, qtAx`) are valued with the same mortality of the table `mt`, changing only the interest rate to $j = (i-q)/(1+q)$. Therefore they respect the percentage `perc` and also work with tables built from `lx` or `qx`. With $q = 0$ they return the same value as the level annuity.

- Fractional payments (`m` > 1) are approximated by adjusting the annual annuity with the factor $\frac{m-1}{2m}$: $\ddot{a}_x^{(m)} = \ddot{a}_x - \frac{m-1}{2m}$ for whole life annuities and $\ddot{a}_{x:\overline{n}|}^{(m)} = \ddot{a}_{x:\overline{n}|} - \frac{m-1}{2m}(1 - {}_nE_x)$ for temporary annuities (with the opposite sign for annuities-immediate). In deferred annuities the adjustment is discounted to the deferring period, so that ${}_{d|}\ddot{a}_x^{(m)} = {}_dE_x \cdot \ddot{a}_{x+d}^{(m)}$. For instance, `annuity(mt, 60, 'w', 0, 12, deferred=5)` returns 11.034774901390385, the same value as `nEx(mt, 60, 5) * annuity(mt, 65, 'w', 0, 12)`.

Examples:

1) The present value of a 5-year (financial) annuity with nominal annual interest rate 12% and monthly payments of $100 is:

```
from pyliferisk import *
from pyliferisk.mortalitytables import FIN
import numpy as np

mt = Actuarial(nt=FIN, i=0.12/12) #FIN = Financial table

n = 5 * 12
C = 100

print(annuity(mt, 0, n, 1) * C) #replace age 'x' by 0
```

Returns:

```
4495.5038406224
```

The equivalent formula in Excel is: `PV(12%/12,12*20,500,,0)`

2) Premium calculation: A Life Temporal insurance for a male, 30 years-old and a horizon of 10 years, fixed annual premium (GKM95, interest 6%):

Actuarial equivalence: $\pi^{1}_{30:\overline{10|}} = 1000 \cdot \dfrac{A^{1}_{30:\overline{10|}}}{\ddot{a}_{30:\overline{10|}}}$

```
from pyliferisk import *
from pyliferisk.mortalitytables import GKM95

nt = Actuarial(nt=GKM95, i=0.06)
x = 30
n = 10
C = 1000

print(C * (Axn(nt, x, n) / annuity(nt, x, n, 0)))
```

```
1.3581166922356984
```

3) Reserving a life risk insurance with regular premium. Applying the equivalence principle, where:
${}_0V_x = A_x - \pi \cdot \ddot{a}_x = 0$ and: $\pi = \dfrac{A^{1}_{x:\overline{n|}}}{\ddot{a}_{x:\overline{n|}}}$. At time t: ${}_tV_x = A^{\ 1}_{40+t:\overline{10-t|}} - \pi \cdot \ddot{a}_{40+t:\overline{10-t|}}$

```
from pyliferisk import *
from pyliferisk.mortalitytables import GKM95

nt = Actuarial(nt=GKM95, i=0.03)
x = 40
n = 20
Cm = 100000
Premium = Cm * Axn(nt, x, n) / annuity(nt, x, n, 0)  #fixed premium

def Reserve(t):
    return round(Cm * Axn(nt, x+t, n-t) - Premium * annuity(nt, x+t, n
        -t, 0),2)

for t in range(0, n+1):
    print(t, Reserve(t))
```

```
0 0.0
1 253.34
2 501.92
3 743.94
4 977.36
5 1199.79
6 1408.34
7 1599.67
8 1769.94
9 1914.83
10 2029.44
11 2108.34
12 2145.56
13 2134.42
14 2067.62
15 1937.21
```

```
16 1734.56
17 1450.27
18 1074.07
19 594.76
20 0.0
```

Actuarial notation vs. Syntax formula

Notation	Description	Syntax
$\ddot{a}_{x:\overline{n}\rceil}$	n-year temporary life annuity-due	`annuity(nt,x,n,0)`
$a_{x:\overline{n}\rceil}$	n-year temporary life annuity	`annuity(nt,x,n,1)`
$\ddot{a}^{(m)}_{x:\overline{n}\rceil}$	n-year annuity-due m-monthly payments	`annuity(nt,x,n,0,m)`
$\ddot{a}_x$	whole life annuity-due	`annuity(nt,x,'w',0)`
a_x	whole life annuity	`annuity(nt,x,'w',1)`
$\ddot{a}^{(m)}_x$	whole life annuity-due m-monthly	`annuity(nt,x,'w',0,m)`
$a^{(m)}_x$	whole life annuity m-monthly	`annuity(nt,x,'w',1,m)`
${}_{d\rceil}\ddot{a}_{x:\overline{n}\rceil}$	d-year deferred n-year temporary life annuity-due	`annuity(nt, x, n, 0, deferred=d)`
${}_{d\rceil}a_{x:\overline{n}\rceil}$	d-year deferred n-year temporary life annuity	`annuity(nt, x, n, 1, deferred=d)`
${}_{d\rceil}\ddot{a}^{(m)}_{x:\overline{n}\rceil}$	d-year deferred n-year temporal annuity-due m-monthly payments	`annuity(nt, x, n, 0, m, deferred=d)`
${}_{d\rceil}\ddot{a}_x$	d-year deferred whole life annuity-due	`annuity(nt, x, 'w', 0, deferred=d)`
${}_{d\rceil}a_x$	d-year deferred whole life annuity	`annuity(nt, x, 'w', 1, deferred=d)`
Increasing annuities (a sample of them)		
${}^{q}\ddot{a}_x$	geometrically increasing whole-life annuity-due	`annuity(nt, x, 'w', 0, increase=['g',c])`
${}^{q}_{d\rceil}\ddot{a}_x$	d-year deferred geometrically increasing whole-life annuity-due	`annuity(nt, x, 'w', 0, deferred=d, increase=['g',c])`
${}^{q}_{d\rceil}\ddot{a}^{(m)}_{x:\overline{n}\rceil}$	d-year deferred geometrically increasing n-year temporal annuity-due m-monthly payments	`annuity(nt, x, n, 0, m, deferred=d, increase=['g',c])`

2.4 Pure endowment: Deferred capital

2.4.1 formula `nEx()`

Description	Returns the EPV of a pure endowment (deferred capital). Pure endowment benefits are conditional on the survival of the policyholder.
Actuarial notation	${}_nE_x$
Usage	`nEx(mt, x, n)`
Args	mt: Mortality table. x: the age as integer number. n: period in years.
Example	`mt = Actuarial(nt=PASEM2020_M_NoRel_1orden, i=0.02)` `nEx(mt, 50, 10)`

Syntax:

Notation	Description	Syntax
$A_{x:\overline{n}\rceil}^{1}$	Pure endowment (Deferred capital)	`nEx(nt, x, n)`
${}_nE_x$	EPV of a pure endowment (deferred capital)	`nEx(nt, x, n)`

Example:

A deferred capital premium calculation:

```
from pyliferisk import *
from pyliferisk.mortalitytables import GKM80

C = 1000
x = 60
n = 25
exp = 0.2 / 100          # expenses over capital
com =  0.10              # commision over premium

mt = Actuarial(nt=GKM80, i=0.025)

def Premium(mt, x, n):
  return (nEx(mt, x, n) + Axn(mt, x, n)) / annuity(mt, x, n, 0) * C

print((Premium(mt, x, n) + C * exp) / (1 - com))
```

Returns:

```
58.22661317638213
```

2.5 Commutation factors

Nowadays, the standard techniques for actuarial calculations use cashflow projections. In fact, the use of interest rates curves is required for the calculation of the technical provisions for insurance obligations (such as risk-free interest rate term structures in Solvency II framework) where commutation factors are not adequate.

Despite commutation factors may seem quite prehistoric, it may be useful for academic purposes or replicating older products exactly "to the letter". For this reason, the pyliferisk library includes a list of commutations factors (`Dx, Nx, Cx, Mx`) in order to facilitate the migration from older actuarial software (such as Cobol or Cactus) to Python, or for simple calculations (where your model shouldn't be subject to future changes). If your goal is to reduce the timing, there are a lot of other aspects where you can save time.

The commutation factors are available as vectors of the `Actuarial` class (`mt.Dx[x]`, `mt.Nx[x]`, `mt.Cx[x]`, `mt.Mx[x]`) and also as functions (`Dx(mt, x)`, `Nx(mt, x)`, `Cx(mt, x)`, `Mx(mt, x)`). Both use exactly the same criterion, so they always return the same value: N_x and M_x are the sums of D_x and C_x from age x up to the ultimate age.

```
from pyliferisk import *
from pyliferisk.mortalitytables import GKM95

tariff = Actuarial(nt=GKM95, i=0.02)

print(tariff.Dx[50])
print(tariff.Nx[50])
print(tariff.Cx[50])
print(tariff.Mx[50])
```

Returns:

```
34857.91799111972
740831.699375189
147.24736396896128
20331.806238665
```

2.6 Other functions

Other functions can be derived from the lx figures. Anyway, the library include the following additional formulas:

- `qx(mt, x)`: Returns the probability that a life aged x dies before 1 year (q_x).
- `px(mt, x)`: Returns the probability of surviving within 1 year ($p_x = 1 - q_x$).
- `tpx(mt, x, t)`: Returns the probability that x will survive within t years (${}_tp_x$).
- `tqx(mt, x, t)`: Returns the probability to die within t years at age x (${}_tq_x$).
- `tqxn(mt, x, n, t)`: Probability that x survives n years and then dies in the subsequent t years (${}_{n|t}q_x$).
- `mx(mt, x)`: Returns the central mortality rate, $m_x = d_x/L_x$, where L_x is approximated as $l_x - 0.5 \cdot d_x$. It is slightly higher than $q_x = d_x/l_x$.

- `ex(mt, x)`: Returns the curtate expectation of life (e_x), without adding 0.5. It returns 0 for an age outside the table.

All these probabilities and rates are expressed per unit (between 0 and 1), the same scale used internally by the library. Therefore they can be combined directly, for instance `px(mt, x)` equals `tpx(mt, x, 1)` (up to floating-point rounding). If the per mille figure is needed for presentation purposes, just multiply the result by 1000 or use `mt.view(var='qx')`:

```
mt = MortalityTable(nt=GKM95)
print(qx(mt, 50))
print(qx(mt, 50) * 1000)
print(px(mt, 50))
print(mx(mt, 50))
print(ex(mt, 50))
```

```
0.0043087
4.3087
0.9956913
0.004318002488661534
27.389458797448576
```

2.7 Help (Documentation strings)

According Python PEP257 convention, formulas includes a documentation string (also known as docstrings):

```
from pyliferisk import *

print(qx.__doc__)
print(annuity.__doc__)
```

Returns:

```
 qx: Returns the probability that a life aged x dies before 1 year,
    expressed per unit (between 0 and 1).
    Args:
        mt: the mortality table
        x: the age as integer number.

Return the actuarial present value of a life annuity.

    Preferred syntax:
        annuity(mt, x, n, p, m=1, deferred=0, increase=None)
    Args:
        mt = the mortality table (Actuarial instance)
        x = the age as integer number.
        n = An integer number (term of the annuity in years) or 'w' = whole-life.
            (Also, 99 years is defined to be whole-life).
        p = Moment of payment: 0 = beginning of each period (annuity-due),
            1 = end of each period (annuity-immediate).
        m = Payable 'm' per year (fractional payments). Default = 1 (annually)
        deferred = The deferring period in years (keyword argument, default 0).
        increase = None (level annuity), ['g', q] (geometrically increasing at
            rate q) or ['a', q] (arithmetically increasing, unitary).

    Legacy (deprecated) positional syntax is still accepted:
        annuity(mt, x, n, p, m, ['a/g', q], -d)
    where the deferring period is passed as a negative number.
```

3 Mortality tables

The package includes a sample of life mortality tables (`mortalitytables.py`), mainly extracted from academic textbooks or contributions. It's possible to add tables as a list or to import tables of an external source, i.e. txt or csv files (see Example 6).

Tables must be imported for use, either all (`import *`) or only which you will use, example:

```
from pyliferisk.mortalitytables import GKM95, UK43
```

Notes:

- The source tables are expressed per mille (qx * 1000). The classes `MortalityTable` and `Actuarial` convert them to per unit when the table is loaded, so `mt.qx[x]` is the true probability.
- The first item indicates the age when the table starts. For example, UK43 table is 0 for the first 30 ages.

There is a financial table (called `FIN`) for financial annuities which doesn't include mortality.

In the SOA repository (`http://mort.soa.org`) is available a variety (over 2,500) of rate tables of interest to actuaries: SOA experience mortality and lapse tables, regulatory valuation tables, population tables and various international tables).

3.1 Adding tables

3.1.1 Probability of dying tables (qx)

Probability of dying between age x and x+1 tables (qx tables of mortality tables) are added in list format where first item indicates the age when table starts. ie:

SCOT_DLT_00_02_M = [0, 6.205, 0.328, 0.26 ....]

The values of these source tables are expressed per mille (qx * 1000); they are converted to per unit when the table is loaded with `nt=`.

3.1.2 Number of surviving tables (lx)

In case you need to use an surviving table (with numbers of surviving to age x) it's possible using the following instruction:

```
from pyliferisk import MortalityTable

lx_sample = [100000,99500,99250,99200,99000,98900]
mt = MortalityTable(lx=lx_sample)

print(mt.qx[0])
print(mt.qx[1])
print(mt.qx[2])
```

```
0.005
0.002512562814070352
0.0005037783375314861
```

See Example 6 where an external file is read.

3.2 Class .view()

Instruction `.view()` provides a view of the main variables (`qx, lx, dx, ex, Dx, Nx, Cx, Mx`) of any mortality table (the commutation vectors `Dx, Nx, Cx, Mx` are only filled in the `Actuarial` class). The default view is lx for 10-years. Usage example:

```
mt = MortalityTable(nt=INM05)
mt.view()
```

```
[x=0]  lx=100000.0
[x=1]  lx=99564.1
[x=2]  lx=99530.7460265
[x=3]  lx=99506.659586
[x=4]  lx=99488.4498673
[x=5]  lx=99473.4271113
[x=6]  lx=99460.6945127
[x=7]  lx=99448.1624651
[x=8]  lx=99435.5325485
[x=9]  lx=99421.9098806
[x=10]  lx=99409.3827199
Total number of rows for lx = 107
```

Other view (qx from 60 to 65 years-old). The probabilities are stored per unit, but `.view()` shows them per mille as presentation:

```
mt = MortalityTable(nt=INM05)
mt.view(60, 65, var = 'qx')
```

```
[x=60]  qx=9.958
[x=61]  qx=10.736
[x=62]  qx=11.199
[x=63]  qx=12.455
[x=64]  qx=13.861
[x=65]  qx=15.224
Total number of rows for qx = 106
```

4 Examples

This section includes a list of examples using `pyliferisk`.

4.1 Example 1. Cashflow calculation: Annuity-immediate Geometrically increasing

Prospective Reserves look forward, as the expected present value of future outgo less the expected present value of the future income. Using NumPy library for discount them.
Life annuity immediate geometrically increasing for a male 67 years-old, as single premium (PASEM2020, interest 5%):

```
1  import pyliferisk as life
2  from pyliferisk.mortalitytables import PASEM2020_M_NoRel_1orden
3  import numpy as np
4
5  mt = life.MortalityTable(nt=PASEM2020_M_NoRel_1orden)
6
7  age = 67
8  initial_payment = 8000
9  incr = 0.03                                  # increment
10 i = 0.05                                     # interest rate
11
12 discount_factor = []
13 for y in range(0, mt.w-age):
14         discount_factor.append(1 / (1 + i) ** (y + 1))
15
16 payments = [initial_payment]
17 for x in range(0, mt.w-age-1):
18         payments.append(payments[x] * (1 + incr) * (1 - mt.qx[age+x])
               )
19
20 print('Premium:', np.dot(payments, discount_factor).round(2))
```

```
Premium: 127203.85
```

There are different ways of discounting (see previous line 15). In case of:

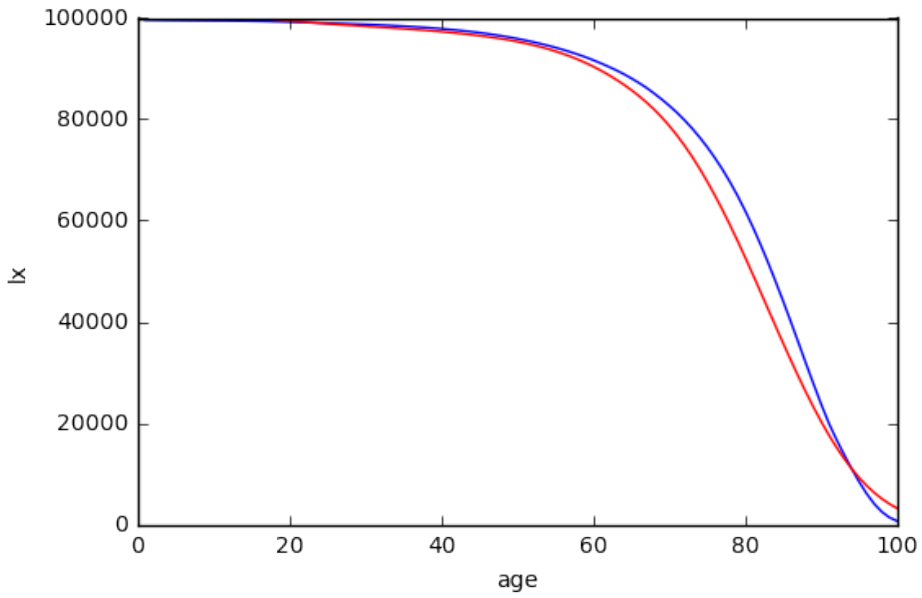
- not discount first year: the exponent should be only `y`.
- discount first year: the exponent should be `y+1`.
- discount half year: the exponent should be `y+0.5`.

4.2 Example 2. Drawing graph with matplotlib

Plotting a surviving graph with pyplot library:

```
import matplotlib.pyplot as plt
import pyliferisk as life
from pyliferisk.mortalitytables import PASEM2020_M_NoRel_1orden,
    GKM95

tariff = life.MortalityTable(nt=PASEM2020_M_NoRel_1orden)
experience = life.MortalityTable(nt=GKM95, perc=75)
x = range(0, tariff.w)
y = tariff.lx[:tariff.w]
z = experience.lx[:tariff.w]
plt.plot(x,y, color = 'blue')
plt.plot(x,z, color = 'red')
plt.ylabel('lx')
plt.xlabel('age')
```



4.3 Example 3. Obtain data from plain text file

Term Life with maturity capital benefit and similar death capital.

```
from pyliferisk import *
from pyliferisk.mortalitytables import GKM80
import csv

mt = Actuarial(nt=GKM80, i=0.04)

def single_risk_premium(x, n):
   return nEx(mt, x, n) + Axn(mt, x, n)

def annual_risk_premium(x, n):
   return (single_risk_premium(x, n) / annuity(mt, x, n, 0))

SingleRiskPrem=[]
AnnualRiskPrem=[]

columns = '{0:8} {1:2} {2:9} {3:8} {4:10} {5:10}'
print(columns.format('Contract', 'Age', 'Duration', 'Capital', '
    Single Pr', 'Annual Pr'))
print('--' * 26)

with open('colective.csv', 'r') as file:
    colective = csv.DictReader(file, delimiter=';')
    for row in colective:
        age = int(row['age'])
        dur = int(row['duration'])
        capital = int(row['capital'])
        single_premium = round(capital * single_risk_premium(age, dur
            ), 2)
        annual_premium = round(capital * annual_risk_premium(age, dur
            ), 2)
        AnnualRiskPrem.append(annual_premium)
        print(columns.format(row['N_pol'], age, dur, capital,
            single_premium, annual_premium))

print('--'*26)
print('Total Annual Premium:', sum(AnnualRiskPrem))
```

```
Contract Age Duration  Capital  Single Pr  Annual Pr
----------------------------------------------------
00001    42       10  2000000 1360835.59   163775.8
00002    35       15  1500000  841999.04   73824.92
00003    51       12  1000000  641866.93   68933.01
00004    31        5  3500000 2878308.37  623242.41
00005    37       20  2000000  940876.05   68334.85
----------------------------------------------------
Total Annual Premium: 998110.99
```

For example, this run spent 0m0.024s in a MacBook Pro 2,6 GHz Intel Core i5 8 GB 1600 MHz DDR3.

4.4 Example 4. Obtain the Risk-free rate from MS Excel

Obtain the risk-free rate from the EIOPA Excel file at 31-12-2025 with pandas. `Pandas` is an open source library providing high-performance, easy-to-use data structures, and data analysis tools. `http://pandas.pydata.org`

```
import pandas as pd

df = pd.read_excel('EIOPA_RFR_20251231_Term_Structures.xlsx',
    sheet_name='RFR_spot_no_VA', skiprows=9, usecols='C:C', names=['
    Euro'])
df['Disc_factor'] = (1 + df['Euro']) ** df.index

print(df.head(20))
```

```
      Euro  Disc_factor
0   0.02076     1.000000
1   0.02163     1.021630
2   0.02283     1.046181
3   0.02386     1.073301
4   0.02479     1.102909
5   0.02565     1.135000
6   0.02651     1.169982
7   0.02724     1.206989
8   0.02793     1.246546
9   0.02863     1.289237
10  0.02921     1.333644
11  0.02975     1.380543
12  0.03033     1.431252
13  0.03076     1.482683
14  0.03107     1.534738
15  0.03133     1.588418
16  0.03156     1.644038
17  0.03176     1.701522
18  0.03194     1.761084
19  0.03209     1.822359
```

4.5 Example 5. Cashflow Calculation Reserving

Reserving for a Whole Life contract using a lineal interest rate and the risk-free rate obtained in the previous example.

Once again, the following two examples should be enough clear:

Using lineal interest rate

```
from pyliferisk import *
from pyliferisk.mortalitytables import INM05
import numpy as np

tariff  = Actuarial(nt=INM05, i=0.05)
reserve = MortalityTable(nt=INM05)
age = 32                          # age
Cd = 3000                         # capital death
Premium = Cd * Ax(tariff, 25) / annuity(tariff, 25, 'w', 0) #fixed at
    age 25

qx_vector = []
px_vector=[]
for i in range(age, reserve.w + 1):
        qx = ((reserve.lx[i] - reserve.lx[i+1]) / reserve.lx[age])
        qx_vector.append(qx)
        qx_sum = sum(qx_vector)
        px_vector.append(1 - qx_sum)

def Reserve(i):
        discount_factor = []
        for y in range(0, reserve.w-age + 1):
                discount_factor.append(1 / ( 1 + i) ** (y + 1))

        APV_Premium = np.dot(Premium, px_vector)
        APV_Claims = np.dot(Cd, qx_vector)
        # Reserve = APV(Premium) - APV(Claim)
        return np.dot(discount_factor, np.subtract(APV_Claims,
           APV_Premium)).round(2)

print(Reserve(0.0191))
print(Reserve(0.0139))
```

```
831.91
1100.77
```

Including risk free rate curve

```
from pyliferisk import *
from pyliferisk.mortalitytables import INM05
import numpy as np
import pandas as pd

rfr = pd.read_excel('EIOPA_RFR_20251231_Term_Structures.xlsx',
    sheet_name='RFR_spot_no_VA',
                                        skiprows=9, usecols='C:C',
                                            names=['Euro'])

tariff  = Actuarial(nt=INM05, i=0.05)
reserve = MortalityTable(nt=INM05)
x = 32                  # age
Cd = 3000               # capital death
Premium = Cd * Ax(tariff, 25) / annuity(tariff, 25, 'w', 0) #fixed at
     age 25

qx_vector = []
px_vector = []
for i in range(x,reserve.w + 1):
        qx = ((reserve.lx[i]-reserve.lx[i+1]) / reserve.lx[x])
        qx_vector.append(qx)
        qx_sum = sum(qx_vector)
        px_vector.append(1 - qx_sum)

def Reserve(i):
        discount_factor = []
        for y in range(0, reserve.w - x + 1):
                if isinstance(i,float):
                        discount_factor.append(1 / (1 + i) ** (y + 1)
                            )
                elif i == 'rfr':
                        discount_factor.append(1 / (1 + rfr['Euro'][y
                            ]) ** (y + 1))

        APV_Premium = np.dot(Premium, px_vector)
        APV_Claims = np.dot(Cd, qx_vector)
        return np.dot(discount_factor, np.subtract(APV_Claims,
            APV_Premium)).round(2)

print(Reserve(0.0191))
print(Reserve(0.0139))
print(Reserve('rfr'))
```

```
831.91
1100.77
376.72
```

4.6 Example 6. Import a mortality table (lx) using pandas

Normally, we will have the tables in an external file in xls or csv format. In this case, we can use the instruction read_csv or read_xls of pandas.

Example 3.3.1 from Actuarial Mathematics, by Newton L. Bowers (et al.), Society of Actuaries, 1997 (pg. 64)

```
import pandas as pd
from pyliferisk import MortalityTable

USLife1979 = pd.read_csv('uslife79.csv')
USLife79lx = USLife1979['lx'].tolist()

mt = MortalityTable(lx=USLife79lx)

print('Live to 100:', mt.lx[100]/mt.lx[20])
print('Die before 70:', 1 - mt.lx[70]/mt.lx[20])
print('Die in the tenth decade of life:', (mt.lx[90]-mt.lx[100])/mt.
    lx[20])


print('Probability of death between 50 and 51:',
        (mt.lx[50] - mt.lx[51]) / mt.lx[50])
print('Likewise with qx:', mt.qx[50])
print('Ultimate age:', mt.w)
```

```
Live to 100: 0.011765789177520181
Die before 70: 0.30174645235878494
Die in the tenth decade of life: 0.13304549779519342
```

Additional exercises not included in book:

```
print('Probability of death between 50 and 51:',
        (mt.lx[50] - mt.lx[51]) / mt.lx[50])
print('Likewise with qx:', mt.qx[50])
print('Ultimate age:', mt.w)
```

```
Probability of death between 50 and 51: 0.005899628520857465
Likewise with qx: 0.005899628520857465
Ultimate age: 109
```

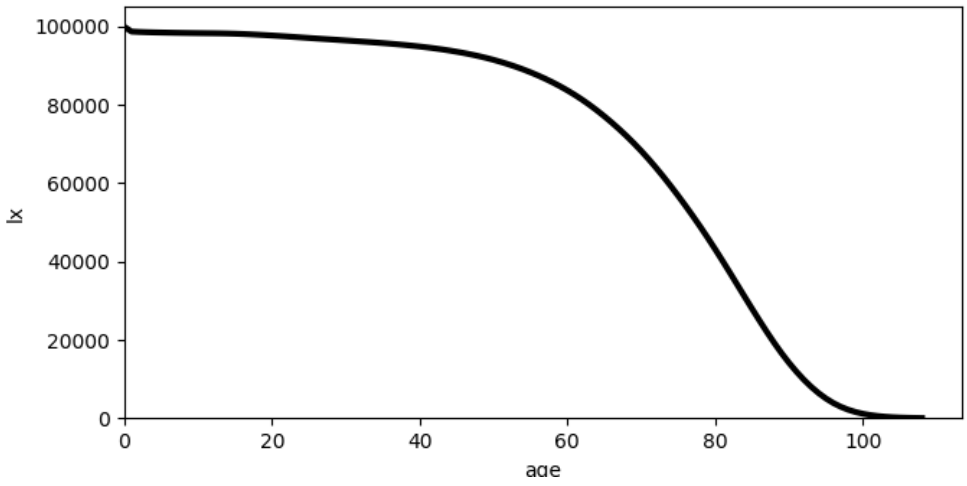
Figure 3.3.3 from Actuarial Mathematics, by Newton L. Bowers (et al.), 1997 (pg. 65)

```
#!/usr/bin/python
import pandas as pd
from pyliferisk import MortalityTable
import matplotlib.pyplot as plt

USLife1979 = pd.read_csv('uslife79.csv')
USLife79lx = USLife1979['lx'].tolist()

mt = MortalityTable(lx=USLife79lx)

x = range(0, mt.w)
y = mt.lx[:mt.w]
plt.plot(x, y, linewidth=3, color='0')
plt.ylabel('lx')
plt.xlabel('age')
plt.show()
```



4.7 Example 7. Cashflows in pandas

Replicating Example 1 with Pandas.

Life annuity immediate geometrically increasing for a male 67 years-old, as single premium (PASEM2020, interest 5%).

```
from pyliferisk import *
from pyliferisk.mortalitytables import PASEM2020_M_NoRel_1orden
import pandas as pd

mt = MortalityTable(nt=PASEM2020_M_NoRel_1orden)

age = 67
init_payment = 8000
incr = 0.03             # increment annutie
i = 0.05                # interest rate

period = len(range(0, mt.w - age))

df = pd.DataFrame(pd.date_range('1/1/2016', periods=period, freq='YE'
    ), columns=['date'])
df['t'] = df.index
df['age'] = pd.Series(list(range(age, mt.w)))
df['px'] = df['t'].apply(lambda t: 1 - mt.qx[age+t])
df['px_cumprod'] = df['px'].cumprod()
df['disc_factor'] = df['t'].apply(lambda t: 1 / (1 + i) ** (t + 1))
df['capital'] = df['t'].apply(lambda t: init_payment * ((1 + incr) **
     t))
df['payments'] = df['t'].apply(lambda t: init_payment if t == 0 else
df['capital'][t] * df['px_cumprod'][t-1])
df['apv_payments'] = df['payments'] * df['disc_factor']
premium = df['apv_payments'].sum().round(2)

print(df.head())
print('Premium:', premium)
```

```
        date  t  age  ...     capital     payments  apv_payments
0 2016-12-31  0   67  ...  8000.00000  8000.000000   7619.047619
1 2017-12-31  1   68  ...  8240.00000  8158.866133   7400.332094
2 2018-12-31  2   69  ...  8487.20000  8313.657554   7181.649976
3 2019-12-31  3   70  ...  8741.81600  8463.121825   6962.631270
4 2020-12-31  4   71  ...  9004.07048  8605.787667   6742.859820

[5 rows x 9 columns]
Premium: 127203.85
```

Export the DataFrame to a MS Excel file:

```
df.to_excel('output.xlsx')
```

H6 | 9004,07048

	A	B	C	D	E	F	G	H	I	J
1		date	t	age	px	px_cumprod	disc_factor	capital	payments	apv_payments
2	0	31/12/23	0	67	0,98434	0,98434	0,95238	8000,00	8000,00	7619,05
3	1	31/12/24	1	68	0,98244	0,96705	0,90703	8240,00	8110,93	7356,85
4	2	31/12/25	2	69	0,98019	0,94789	0,86384	8487,20	8207,54	7089,98
5	3	31/12/26	3	70	0,97754	0,92661	0,82270	8741,82	8286,32	6817,18
6	4	31/12/27	4	71	0,9744	0,90288	0,78353	9004,07	8343,22	6537,13
7	5	31/12/28	5	72	0,97065	0,87638	0,74622	9274,19	8373,48	6248,42
8	6	31/12/29	6	73	0,96617	0,84673	0,71068	9552,42	8371,51	5949,48
9	7	31/12/30	7	74	0,9608	0,81353	0,67684	9838,99	8330,93	5638,70
10	8	31/12/31	8	75	0,95436	0,77641	0,64461	10134,16	8244,47	5314,46
11	9	31/12/32	9	76	0,94666	0,73499	0,61391	10438,19	8104,26	4975,31
12	10	31/12/33	10	77	0,93745	0,68901	0,58468	10751,33	7902,10	4620,19
13	11	31/12/34	11	78	0,92647	0,63835	0,55684	11073,87	7630,02	4248,68
14	12	31/12/35	12	79	0,91345	0,58310	0,53032	11406,09	7281,03	3861,29
15	13	31/12/36	13	80	0,90319	0,52665	0,50507	11748,27	6850,41	3459,92
16	14	31/12/37	14	81	0,89182	0,46968	0,48102	12100,72	6372,81	3065,43
17	15	31/12/38	15	82	0,87931	0,41299	0,45811	12463,74	5853,91	2681,74
18	16	31/12/39	16	83	0,86558	0,35748	0,43630	12837,65	5301,83	2313,17
19	17	31/12/40	17	84	0,85052	0,30404	0,41552	13222,78	4726,85	1964,10
20	18	31/12/41	18	85	0,83395	0,25356	0,39573	13619,46	4140,87	1638,68
21	19	31/12/42	19	86	0,81581	0,20685	0,37689	14028,05	3556,88	1340,55
22	20	31/12/43	20	87	0,79608	0,16467	0,35894	14448,89	2988,77	1072,80
23	21	31/12/44	21	88	0,77485	0,12759	0,34185	14882,36	2450,67	837,76
24	22	31/12/45	22	89	0,7523	0,09599	0,32557	15328,83	1955,87	636,77
25	23	31/12/46	23	90	0,72864	0,06994	0,31007	15788,69	1515,53	469,92
26	24	31/12/47	24	91	0,70418	0,04925	0,29530	16262,35	1137,41	335,88
27	25	31/12/48	25	92	0,67915	0,03345	0,28124	16750,22	824,97	232,01
28	26	31/12/49	26	93	0,65376	0,02187	0,26785	17252,73	577,08	154,57
29	27	31/12/50	27	94	0,62817	0,01374	0,25509	17770,31	388,59	99,13
30	28	31/12/51	28	95	0,60245	0,00828	0,24295	18303,42	251,42	61,08

5 Changes in version 1.13

The main changes introduced in version 1.13 are:

- Breaking change: probabilities are per unit. `mt.qx[x]`, `qx()` and `px()` return values between 0 and 1, instead of the per mille figures of the previous versions. This change has been introduced because the library is mainly used for academic purposes, and the aim is to align it with the reference bibliography. Code written for version 1.12 or earlier must remove any division by 1000. The per mille figures are only shown by `mt.view(var='qx')`. A `qx` list passed to the classes must be per unit.
- `ex` is the curtate expectation of life (0.5 is no longer added) and `mx()` returns the central mortality rate $d_x/(l_x - 0.5 \cdot d_x)$.
- The class `Actuarial` inherits from `MortalityTable` and the interest rate `i` is mandatory. `MortalityTable` no longer uses `i`.
- `annuity()` has the named parameters `deferred` and `increase`. The former positional syntax is deprecated. Invalid arguments raise a `ValueError`.
- Deferred temporary annuities (level and geometrically increasing) are available, and the deferred arithmetically increasing whole life annuity has been corrected.
- Two tables built from different `lx` lists no longer share their `qx` values (issue #18, reported by Xamit Kadirbekov).
- Geometrically increasing annuities and insurances keep the mortality of the table: they respect `perc` and work with tables built from `lx` or `qx` (issue #17, reported and patched by Xamit Kadirbekov).
- The fractional adjustment (`m` > 1) of deferred whole life annuities has been corrected.
- The functions `Nx()`, `Mx()` and `Cx()` return the same values as the vectors of the class, `nEx` is no longer listed in `.view()` and the functions not implemented raise a `NotImplementedError`.
- Python 3.8 or later is required.

6 Books

The author has checked the library with examples from the following textbooks:

- Actuarial Mathematics for Life Contingent Risks (David C. M. Dickson, Mary R. Hardy and Howard R. Waters) Cambridge University Press, 2009.
- Actuarial Mathematics (2nd Edition), Newton L. Bowers (et al.), Society of Actuaries, 1997.
- Matemática de los Seguros de Vida, (Gil Fana J.A., Heras Matínez A. and Vilar Zanón J.L.) Fundación Mapfre Estudios, 1999.

It will be documented in the examples folder. Contributions are greatly appreciated.

7 Acknowledgements

My special thanks for all the contributions, suggestions and discussion to Florian Pons (France), Mason Borda (US), Sunil Subbakrishna (US), Arief Anbiya (Indonesia), Marios Cheristanidis, Amit Agarwal and Xamit Kadirbekov (GERO Research).